토큰 브리징 시스템 분석

1. 토큰 브리징 개요

MiniEVM의 토큰 브리징 시스템은 Cosmos 네이티브 토큰과 EVM 토큰 간의 양방향 변환을 제공합니다.

1.1 지원하는 토큰 타입

- **ERC20 ↔ Cosmos Coin**: 대체 가능한 토큰
- **ERC721 ↔ Cosmos NFT**: 대체 불가능한 토큰 (NFT)

1.2 브리징 원리

- **Lock & Mint**: 원본 토큰을 잠그고 대상 체인에서 새 토큰 발행
- **Burn & Unlock**: 브리지된 토큰을 소각하고 원본 토큰 잠금 해제
- **1:1 페깅**: 항상 1:1 비율로 토큰 변환 보장

2. ERC20 토큰 브리징

2.1 ERC20 Keeper 구조

```
type ERC20Keeper struct {
keeper *Keeper
// ERC20 팩토리 컨트랙트 ABI
factoryABI *abi.ABI
```

```
// ERC20 래퍼 컨트랙트 ABI
wrapperABI *abi.ABI
// 표준 ERC20 ABI
erc20ABI *abi.ABI
}
```

2.2 ERC20 팩토리 시스템

팩토리 컨트랙트 배포

```
func (k *ERC20Keeper) DeployERC20Factory(ctx context.Context) (common.Address, e
// 1. 팩토리 컨트랙트 바이트코드 준비
factoryBytecode := contracts.ERC20FactoryBytecode
// 2. 컨트랙트 배포
factoryAddr, err := k.keeper.DeployContract(ctx, factoryBytecode, nil)
if  err != nil {
return common.Address{}, err
}
// 3. 팩토리 주소 저장
err = k.keeper.ERC20FactoryAddr.Set(ctx, factoryAddr.Bytes())
if  err != nil {
return common.Address{}, err
}
return  factoryAddr, nil
```

```
}
```

ERC20 토큰 생성

```
func (k *ERC20Keeper) CreateERC20(ctx context.Context, denom  string) (common.Ad

// 1. 이미 존재하는지 확인

if  addr, err := k.keeper.GetContractAddrByDenom(ctx, denom);  err == nil {

return  addr, nil

}

// 2. 메타데이터 조회

metadata, found := k.keeper.bankKeeper.GetDenomMetaData(ctx, denom)

if !found {

return common.Address{}, types.ErrDenomNotFound

}

// 3. 팩토리 컨트랙트 호출

factoryAddr, err := k.keeper.GetERC20FactoryAddr(ctx)

if  err != nil {

return common.Address{}, err

}

// 4. createERC20 함수 호출 데이터 생성

callData, err := k.factoryABI.Pack(

"createERC20",

metadata.Name,
```

```go
metadata.Symbol,
uint8(metadata.DenomUnits[len(metadata.DenomUnits)-1].Exponent),
)
if err != nil {
return common.Address{}, err
}
// 5. 컨트랙트 호출 실행
result, err := k.keeper.CallContract(ctx, factoryAddr, callData)
if err != nil {
return common.Address{}, err
}
// 6. 생성된 컨트랙트 주소 추출
var contractAddr common.Address
err = k.factoryABI.UnpackIntoInterface(&contractAddr, "createERC20", result.Ret)
if err != nil {
return common.Address{}, err
}
// 7. 매핑 정보 저장
err = k.keeper.ERC20DenomsByContractAddr.Set(ctx, contractAddr.Bytes(), denom)
if err != nil {
return common.Address{}, err
}
err = k.keeper.ERC20ContractAddrsByDenom.Set(ctx, denom, contractAddr.Bytes())
```

```go
if err != nil {
return common.Address{}, err
}
// 8. ERC20 등록
err = k.keeper.ERC20s.Set(ctx, contractAddr.Bytes())
if err != nil {
return common.Address{}, err
}
return contractAddr, nil
}
```

2.3 Cosmos Coin → ERC20 변환

```go
func (k *ERC20Keeper) ConvertCoinToERC20(ctx context.Context, sender sdk.AccAddr
// 1. ERC20 컨트랙트 주소 조회 또는 생성
contractAddr, err := k.keeper.GetContractAddrByDenom(ctx, coin.Denom)
if err != nil {
// 컨트랙트가 없으면 새로 생성
contractAddr, err = k.CreateERC20(ctx, coin.Denom)
if err != nil {
return err
}
}
```

```go
// 2. Cosmos 토큰을 모듈 계정으로 전송 (Lock)
moduleAddr := k.keeper.accountKeeper.GetModuleAddress(types.ModuleName)
err = k.keeper.bankKeeper.SendCoins(ctx, sender, moduleAddr, sdk.NewCoins(coin))
if err != nil {
return err
}
// 3. ERC20 토큰 민팅
mintData, err := k.erc20ABI.Pack("mint", receiver, coin.Amount.BigInt())
if err != nil {
return err
}
_, err = k.keeper.CallContract(ctx, contractAddr, mintData)
if err != nil {
// 실패 시 Cosmos 토큰 환불
k.keeper.bankKeeper.SendCoins(ctx, moduleAddr, sender, sdk.NewCoins(coin))
return err
}
// 4. 이벤트 발생
ctx.EventManager().EmitEvent(
sdk.NewEvent(
types.EventTypeCoinToERC20,
sdk.NewAttribute(types.AttributeKeySender, sender.String()),
sdk.NewAttribute(types.AttributeKeyReceiver, receiver.Hex()),
```

```
sdk.NewAttribute(types.AttributeKeyDenom, coin.Denom),
sdk.NewAttribute(types.AttributeKeyAmount, coin.Amount.String()),
sdk.NewAttribute(types.AttributeKeyContractAddr, contractAddr.Hex()),
),
)
return nil
}
```

2.4 ERC20 → Cosmos Coin 변환

```
func (k *ERC20Keeper) ConvertERC20ToCoin(ctx context.Context, sender common.Addr
// 1. 컨트랙트가 등록된 ERC20인지 확인
denom, err := k.keeper.GetDenomByContractAddr(ctx, contractAddr)
if err != nil {
return types.ErrUnregisteredERC20
}
// 2. ERC20 토큰 소각 (Burn)
burnData, err := k.erc20ABI.Pack("burnFrom", sender, amount)
if err != nil {
return err
}
_, err = k.keeper.CallContract(ctx, contractAddr, burnData)
```

```
if  err != nil {
return  err
}
// 3. Cosmos 토큰 잠금 해제 (Unlock)
moduleAddr := k.keeper.accountKeeper.GetModuleAddress(types.ModuleName)
coin := sdk.NewCoin(denom, sdk.NewIntFromBigInt(amount))
err = k.keeper.bankKeeper.SendCoins(ctx, moduleAddr, receiver, sdk.NewCoins(coin
if  err != nil {
// 실패 시 ERC20 토큰 재발행
mintData, _ := k.erc20ABI.Pack("mint", sender, amount)
k.keeper.CallContract(ctx, contractAddr, mintData)
return  err
}
// 4. 이벤트 발생
ctx.EventManager().EmitEvent(
sdk.NewEvent(
types.EventTypeERC20ToCoin,
sdk.NewAttribute(types.AttributeKeySender, sender.Hex()),
sdk.NewAttribute(types.AttributeKeyReceiver, receiver.String()),
sdk.NewAttribute(types.AttributeKeyDenom, denom),
sdk.NewAttribute(types.AttributeKeyAmount, amount.String()),
sdk.NewAttribute(types.AttributeKeyContractAddr, contractAddr.Hex()),
),
```

```
)
return  nil
}
```

3. ERC721 NFT 브리징

3.1 ERC721 Keeper 구조

```
type ERC721Keeper struct {
keeper *Keeper
// ERC721 팩토리 컨트랙트 ABI
factoryABI *abi.ABI
// 표준 ERC721 ABI
erc721ABI *abi.ABI
}
```

3.2 ERC721 컬렉션 생성

```
func (k *ERC721Keeper) CreateERC721(ctx context.Context, classId  string, classU
// 1. 이미 존재하는지 확인
if  addr, err := k.keeper.GetContractAddrByClassId(ctx, classId);  err == nil {
return  addr, nil
}
```

```go
// 2. NFT 클래스 정보 조회
class, found := k.keeper.nftKeeper.GetClass(ctx, classId)
if !found {
return common.Address{}, types.ErrClassNotFound
}
// 3. 팩토리를 통한 ERC721 컨트랙트 배포
factoryAddr, err := k.keeper.GetERC721FactoryAddr(ctx)
if  err != nil {
return common.Address{}, err
}
// 4. createERC721 함수 호출
callData, err := k.factoryABI.Pack(
"createERC721",
class.Name,
class.Symbol,
classUri,
)
if  err != nil {
return common.Address{}, err
}
result, err := k.keeper.CallContract(ctx, factoryAddr, callData)
if  err != nil {
return common.Address{}, err
```

```
}

// 5. 생성된 컨트랙트 주소 추출 및 저장

var  contractAddr common.Address

err = k.factoryABI.UnpackIntoInterface(&contractAddr, "createERC721", result.Ret

if  err != nil {

return common.Address{}, err

}

// 6. 매핑 정보 저장

err = k.keeper.ERC721ClassIdsByContractAddr.Set(ctx, contractAddr.Bytes(), class

if  err != nil {

return common.Address{}, err

}

err = k.keeper.ERC721ContractAddrsByClassId.Set(ctx, classId, contractAddr.Bytes

if  err != nil {

return common.Address{}, err

}

err = k.keeper.ERC721ClassURIs.Set(ctx, contractAddr.Bytes(), classUri)

if  err != nil {

return common.Address{}, err

}

return  contractAddr, nil

}
```

3.3 Cosmos NFT → ERC721 변환

```
func (k *ERC721Keeper) ConvertNFTToERC721(ctx context.Context, sender sdk.AccAdd

// 1. NFT 소유권 확인

owner := k.keeper.nftKeeper.GetOwner(ctx, classId, tokenId)

if !owner.Equals(sender) {

return  types.ErrNotNFTOwner

}

// 2. ERC721 컨트랙트 주소 조회 또는 생성

contractAddr, err := k.keeper.GetContractAddrByClassId(ctx, classId)

if  err != nil {

// 컨트랙트가 없으면 새로 생성

class, found := k.keeper.nftKeeper.GetClass(ctx, classId)

if !found {

return  types.ErrClassNotFound

}

contractAddr, err = k.CreateERC721(ctx, classId, class.Uri, class.Data)

if  err != nil {

return  err

}

}

// 3. NFT 정보 조회

nft, found := k.keeper.nftKeeper.GetNFT(ctx, classId, tokenId)
```

```go
if !found {
return  types.ErrNFTNotFound
}
// 4. Cosmos NFT를 모듈로 전송 (Lock)
moduleAddr := k.keeper.accountKeeper.GetModuleAddress(types.ModuleName)
err = k.keeper.nftKeeper.Transfer(ctx, classId, tokenId, moduleAddr)
if  err != nil {
return  err
}
// 5. ERC721 토큰 민팅
tokenIdBigInt, ok := new(big.Int).SetString(tokenId, 10)
if !ok {
return  types.ErrInvalidTokenId
}
mintData, err := k.erc721ABI.Pack("mint", receiver, tokenIdBigInt, nft.Uri)
if  err != nil {
return  err
}
_, err = k.keeper.CallContract(ctx, contractAddr, mintData)
if  err != nil {
// 실패 시 NFT 반환
k.keeper.nftKeeper.Transfer(ctx, classId, tokenId, sender)
return  err
```

```
}

// 6. 이벤트 발생

ctx.EventManager().EmitEvent(

sdk.NewEvent(

types.EventTypeNFTToERC721,

sdk.NewAttribute(types.AttributeKeySender, sender.String()),

sdk.NewAttribute(types.AttributeKeyReceiver, receiver.Hex()),

sdk.NewAttribute(types.AttributeKeyClassId, classId),

sdk.NewAttribute(types.AttributeKeyTokenId, tokenId),

sdk.NewAttribute(types.AttributeKeyContractAddr, contractAddr.Hex()),

),

)

return  nil

}
```

3.4 ERC721 → Cosmos NFT 변환

```
func (k *ERC721Keeper) ConvertERC721ToNFT(ctx context.Context, sender common.Add

// 1. 컨트랙트가 등록된 ERC721인지 확인

classId, err := k.keeper.GetClassIdByContractAddr(ctx, contractAddr)

if  err != nil {

return  types.ErrUnregisteredERC721

}
```

```go
// 2. ERC721 소유권 확인
ownerData, err := k.erc721ABI.Pack("ownerOf", tokenId)
if err != nil {
return err
}
result, err := k.keeper.CallContract(ctx, contractAddr, ownerData)
if err != nil {
return err
}
var owner common.Address
err = k.erc721ABI.UnpackIntoInterface(&owner, "ownerOf", result.Ret)
if err != nil {
return err
}
if owner != sender {
return types.ErrNotERC721Owner
}
// 3. 토큰 URI 조회
tokenURIData, err := k.erc721ABI.Pack("tokenURI", tokenId)
if err != nil {
return err
}
result, err = k.keeper.CallContract(ctx, contractAddr, tokenURIData)
```

```go
if err != nil {
return err
}
var tokenURI string
err = k.erc721ABI.UnpackIntoInterface(&tokenURI, "tokenURI", result.Ret)
if err != nil {
return err
}
// 4. ERC721 토큰 소각 (Burn)
burnData, err := k.erc721ABI.Pack("burn", tokenId)
if err != nil {
return err
}
_, err = k.keeper.CallContract(ctx, contractAddr, burnData)
if err != nil {
return err
}
// 5. Cosmos NFT 잠금 해제 (Unlock)
tokenIdStr := tokenId.String()
moduleAddr := k.keeper.accountKeeper.GetModuleAddress(types.ModuleName)
err = k.keeper.nftKeeper.Transfer(ctx, classId, tokenIdStr, receiver)
if err != nil {
// 실패 시 ERC721 토큰 재발행
```

```
mintData, _ := k.erc721ABI.Pack("mint", sender, tokenId, tokenURI)

k.keeper.CallContract(ctx, contractAddr, mintData)

return  err

}

// 6. 이벤트 발생

ctx.EventManager().EmitEvent(

sdk.NewEvent(

types.EventTypeERC721ToNFT,

sdk.NewAttribute(types.AttributeKeySender, sender.Hex()),

sdk.NewAttribute(types.AttributeKeyReceiver, receiver.String()),

sdk.NewAttribute(types.AttributeKeyClassId, classId),

sdk.NewAttribute(types.AttributeKeyTokenId, tokenIdStr),

sdk.NewAttribute(types.AttributeKeyContractAddr, contractAddr.Hex()),

),

)

return  nil

}
```

4. 브리징 메시지 타입

4.1 ERC20 브리징 메시지

```
// Cosmos Coin → ERC20
```

```go
type MsgConvertCoinToERC20 struct {
Sender  string
Receiver  string
Coin sdk.Coin
}

// ERC20 → Cosmos Coin
type MsgConvertERC20ToCoin struct {
Sender  string
Receiver  string
ContractAddr  string
Amount  string
}
```

4.2 ERC721 브리징 메시지

```go
// Cosmos NFT → ERC721
type MsgConvertNFTToERC721 struct {
Sender  string
Receiver  string
ClassId  string
TokenId  string
}
```

```
// ERC721 → Cosmos NFT
type MsgConvertERC721ToNFT struct {
Sender  string
Receiver  string
ContractAddr  string
TokenId  string
}
```

5. 자동 브리징 시스템

5.1 EVM 후크를 통한 자동 변환

```
func (k *Keeper) PostTxProcessing(ctx sdk.Context, msg core.Message, receipt *ty
// 1. ERC20 전송 이벤트 감지
for  _, log := range  receipt.Logs {
if  k.isERC20TransferToCosmos(log) {
err := k.handleERC20ToCosmos(ctx, log)
if  err != nil {
k.logger.Error("failed to handle ERC20 to Cosmos conversion", "error", err)
}
}
// 2. ERC721 전송 이벤트 감지
```

```
if  k.isERC721TransferToCosmos(log) {
err := k.handleERC721ToCosmos(ctx, log)
if  err != nil {
k.logger.Error("failed to handle ERC721 to Cosmos conversion", "error", err)
}
}
}
return  nil
}
```

5.2 특별 주소를 통한 자동 변환

```
const (
// 특별한 주소로 전송 시 자동 변환
AutoConvertAddress = "0x0000000000000000000000000000000000000001"
)

func (k *Keeper) handleAutoConvert(ctx sdk.Context, from common.Address, to comm
if  to != common.HexToAddress(AutoConvertAddress) {
return  nil
}
// Cosmos 주소로 변환 (from 주소 기반)
```

```
cosmosAddr := k.convertEVMToCosmos(from)

// 자동 변환 실행

return k.ConvertERC20ToCoin(ctx, from, cosmosAddr, contractAddr, amount)

}
```

6. 보안 및 검증

6.1 재진입 공격 방지

```
type ReentrancyGuard struct {

mutex sync.Mutex

locks map[string]bool

}


func (rg *ReentrancyGuard) Lock(key string) error {

rg.mutex.Lock()

defer rg.mutex.Unlock()

if rg.locks[key] {

return errors.New("reentrancy detected")

}

rg.locks[key] = true

return nil

}
```

```go
func (rg *ReentrancyGuard) Unlock(key  string) {
rg.mutex.Lock()
defer  rg.mutex.Unlock()
delete(rg.locks, key)
}
```

6.2 토큰 공급량 검증

```go
func (k *Keeper) ValidateTokenSupply(ctx context.Context, denom  string, contrac
// 1. Cosmos 토큰 공급량 조회
cosmosSupply := k.bankKeeper.GetSupply(ctx, denom)
// 2. ERC20 토큰 공급량 조회
totalSupplyData, err := k.erc20ABI.Pack("totalSupply")
if  err != nil {
return  err
}
result, err := k.CallContract(ctx, contractAddr, totalSupplyData)
if  err != nil {
return  err
}
var  erc20Supply *big.Int
err = k.erc20ABI.UnpackIntoInterface(&erc20Supply, "totalSupply", result.Ret)
```

```go
if err != nil {
return err
}
// 3. 공급량 일치 확인
if cosmosSupply.Amount.BigInt().Cmp(erc20Supply) != 0 {
return types.ErrSupplyMismatch
}
return nil
}
```

7. 이벤트 및 모니터링

7.1 브리징 이벤트

```go
const (
EventTypeCoinToERC20 = "coin_to_erc20"
EventTypeERC20ToCoin = "erc20_to_coin"
EventTypeNFTToERC721 = "nft_to_erc721"
EventTypeERC721ToNFT = "erc721_to_nft"
AttributeKeySender = "sender"
AttributeKeyReceiver = "receiver"
AttributeKeyDenom = "denom"
AttributeKeyAmount = "amount"
```

```
AttributeKeyContractAddr = "contract_addr"

AttributeKeyClassId = "class_id"

AttributeKeyTokenId = "token_id"

)
```

7.2 메트릭 수집

```
func (k *Keeper) RecordBridgingMetrics(ctx sdk.Context, eventType  string, amoun

// 브리징 볼륨 추적

k.metrics.BridgingVolume.With(

"type", eventType,

"chain_id", ctx.ChainID(),

).Add(float64(amount.Uint64()))

// 브리징 횟수 추적

k.metrics.BridgingCount.With(

"type", eventType,

"chain_id", ctx.ChainID(),

).Inc()

}
```

이러한 토큰 브리징 시스템을 통해 MiniEVM은 Cosmos와 EVM 생태계 간의 완전한 토큰 상호 운용성을 제공합니다.